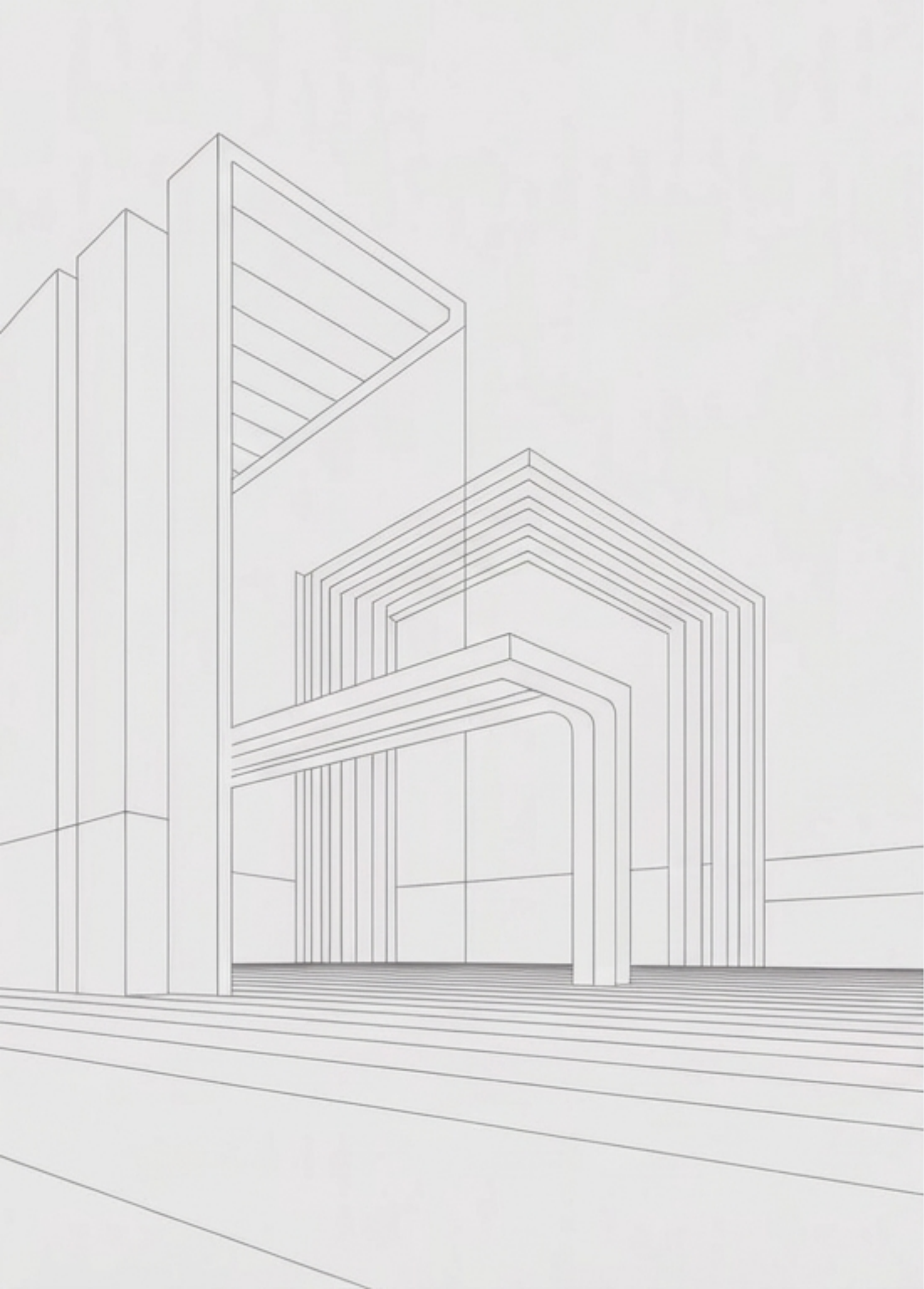




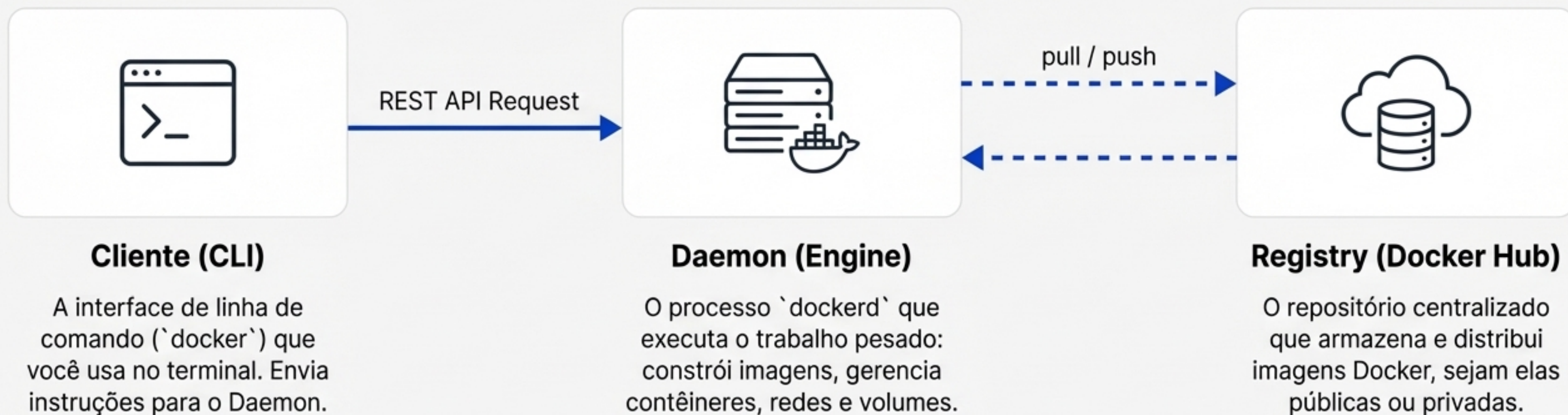
Arquitetura Docker & Setup com WSL2

Do Fundamento à Maestria em Ambiente Windows

Domine a arquitetura cliente-servidor do Docker e configure seu ambiente de desenvolvimento profissional no Windows com WSL2. Aprenda os comandos essenciais para gerenciar contêineres e imagens com confiança.

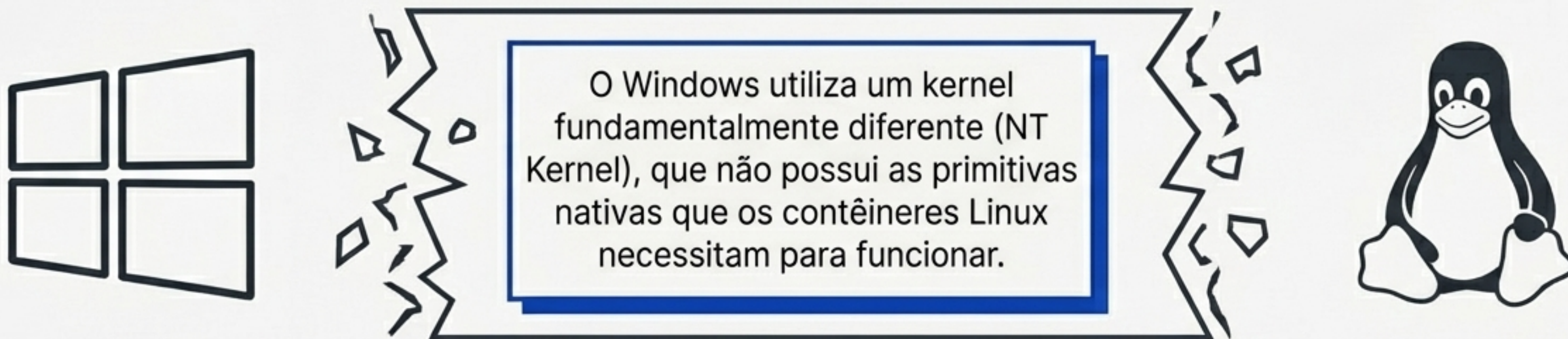
A Arquitetura Cliente-Servidor do Docker

O Docker opera com um modelo que separa a interface de comando (**Cliente**) do motor de execução (**Daemon**), comunicando-se via REST API.



O Desafio no Windows: Um Kernel Incompatível

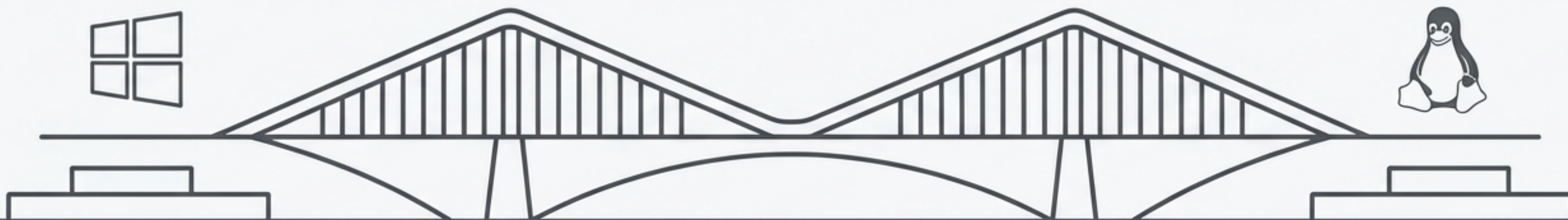
O Docker Engine foi projetado para o ecossistema Linux. Ele depende de funcionalidades específicas do kernel Linux para isolar processos e gerenciar recursos de contêineres.



Uma 'ponte' é necessária para traduzir ou fornecer o ambiente Linux que o Docker Daemon exige.

A Solução: WSL2, A Ponte Perfeita

O Docker Desktop para Windows utiliza o Windows Subsystem for Linux 2 (WSL2) como seu motor. Ele cria uma máquina virtual Linux leve e totalmente integrada, resolvendo o desafio do kernel.



Performance Nativa

Execução de contêineres com desempenho próximo ao nativo, sem a sobrecarga de virtualização tradicional.



Integração Transparente

O cliente Docker no WSL comunica-se perfeitamente com o Daemon, com acesso direto ao sistema de arquivos do Windows.

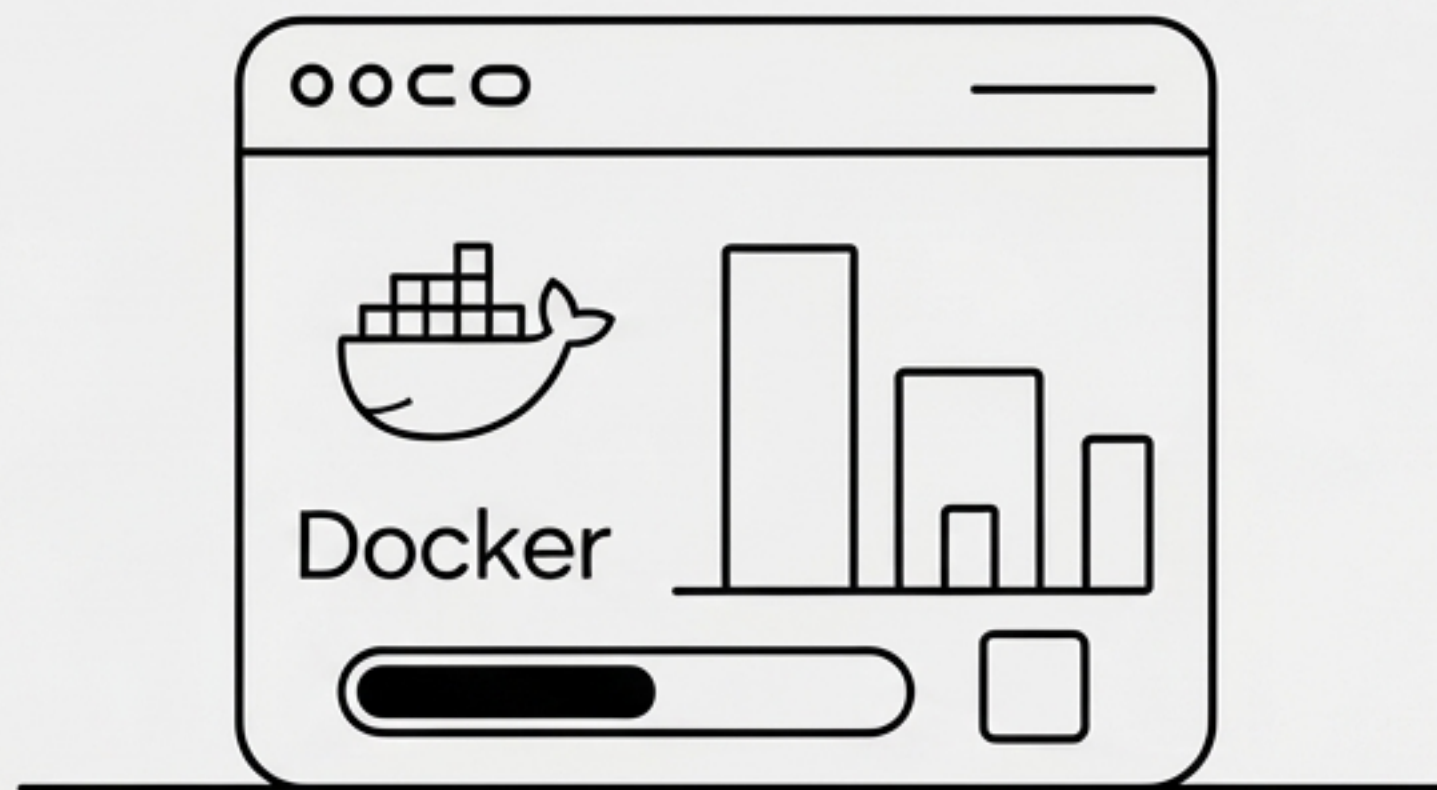


Ferramentas Unificadas

Use suas IDEs e ferramentas favoritas do Windows (como VS Code) para desenvolver dentro de contêineres Linux.

Guia de Instalação e Configuração

- 1. Download:** Baixe o Docker Desktop do site oficial.
- 2. Instalação:** Execute o instalador, garantindo que a opção 'Use WSL 2 instead of Hyper-V' esteja selecionada.
- 3. Reinicialização:** Reinicie o sistema quando solicitado.
- 4. Verificação:** Abra o Docker Desktop e, em `Settings > Resources > WSL Integration`, confirme que a integração com sua distribuição WSL está ativa.



O Primeiro Teste

```
docker run hello-world
```

Uma mensagem de sucesso confirmará que sua instalação está funcionando perfeitamente.

Eliminando a Fricção: Trabalhe sem `sudo`

Por padrão, o Docker requer privilégios de administrador. Para um fluxo de trabalho profissional, configure seu usuário para gerenciar o Docker diretamente.

1. Adicionar ao Grupo Docker

Adicione seu usuário ao grupo `docker`.

```
sudo usermod -aG docker $USER
```

2. Recarregar a Sessão

Para que as alterações

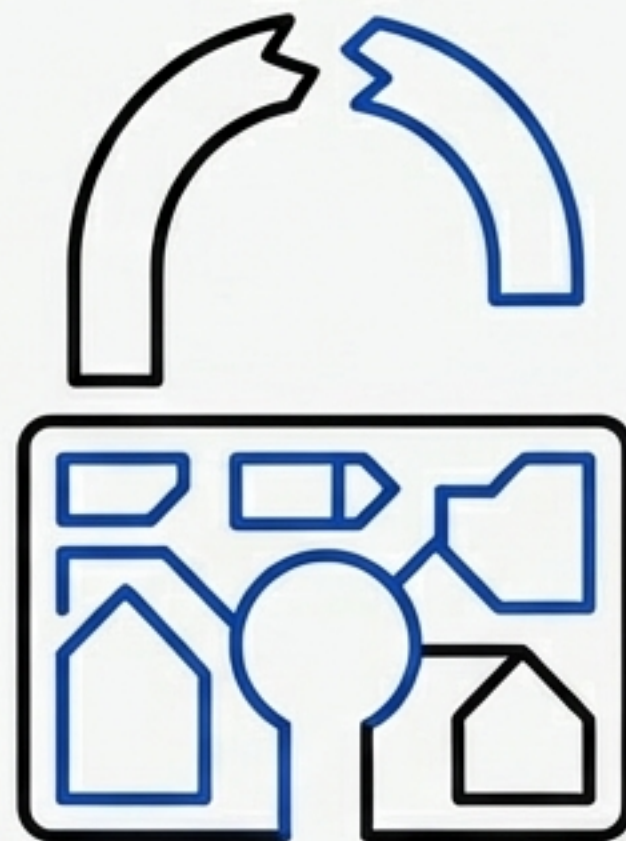
Para que as alterações tenham efeito, feche e reabra seu terminal WSL ou execute o comando:

```
newgrp docker
```

3. Verificar

Teste a configuração executando um comando Docker sem `sudo`.

```
docker run hello-world
```



As Plantas: Gerenciando Imagens Docker

Imagens Docker são templates imutáveis, de leitura, que contêm tudo o que é necessário para executar uma aplicação: código, runtime, bibliotecas e dependências.

Baixar Imagens

```
docker pull [imagem:tag]
```

Obtém uma imagem do Docker Hub. Se a tag for omitida, o padrão é `latest`.

Listar Imagens Locais

```
docker images
```

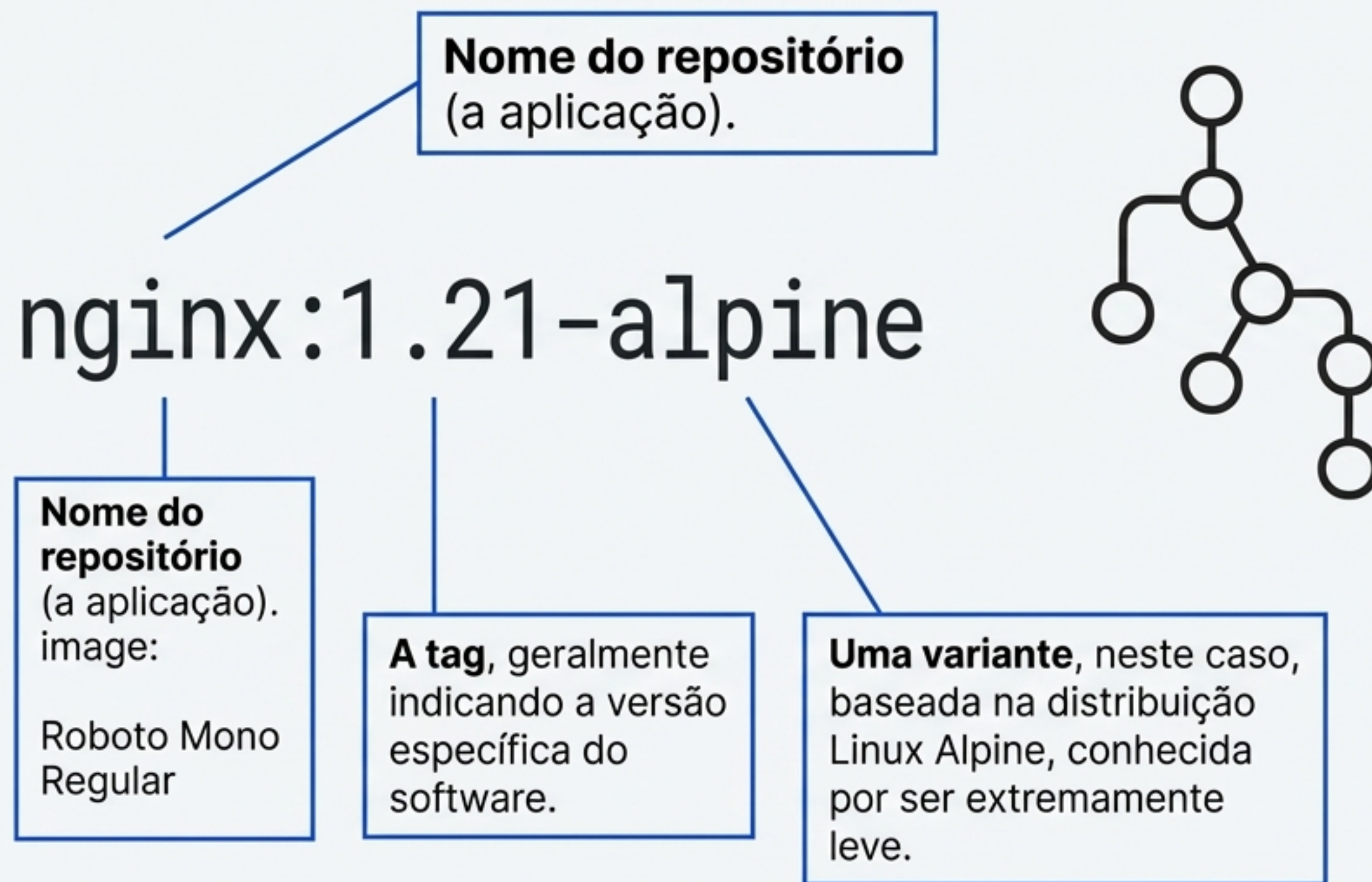
Mostra todas as imagens armazenadas em sua máquina.

Remover Imagens

```
docker rmi [ID ou nome:tag]
```

Exclui uma ou mais imagens para liberar espaço. Use -f para forçar a remoção.

A Anatomia de uma Imagem: Tags e Versionamento

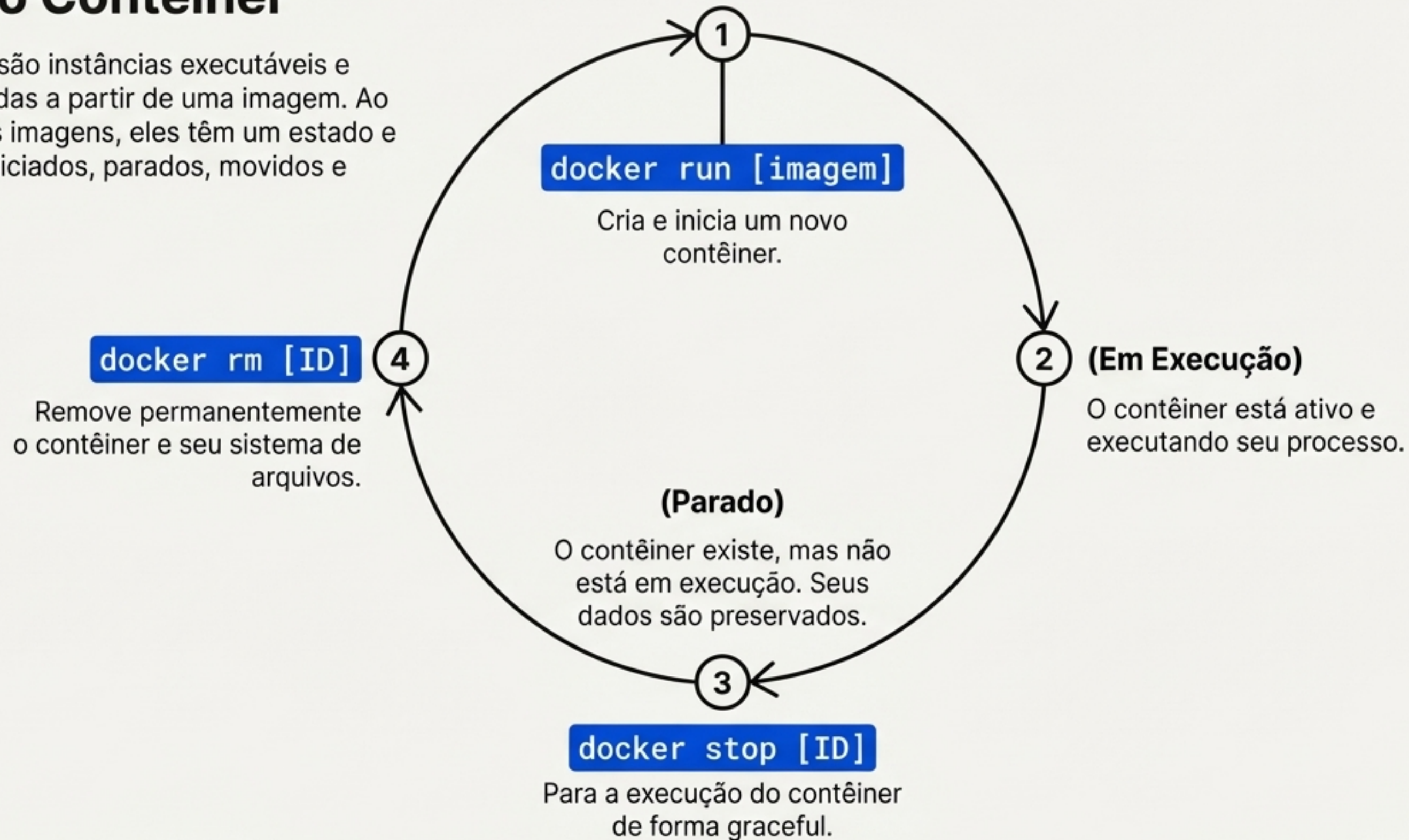


Boas Práticas

- **Seja Específico:** Sempre especifique uma versão exata (``1.21``) em vez de confiar na tag ``latest`` em ambientes de produção para garantir builds reprodutíveis.
- **Prefira Variantes Leves:** Use variantes como ``alpine`` ou ``slim`` sempre que possível para reduzir drasticamente o tamanho da imagem, melhorando a segurança e a velocidade de download.

As Instâncias: O Ciclo de Vida do Contêiner

Contêineres são instâncias executáveis e isoladas criadas a partir de uma imagem. Ao contrário das imagens, eles têm um estado e podem ser iniciados, parados, movidos e excluídos.



Comandos Essenciais de Execução de Contêineres

Executar em background (`-d`)

```
docker run -d [imagem]
```

Libera seu terminal. Essencial para serviços de longa duração como servidores web ou bancos de dados.

Nomear um contêiner (`--name`)

```
docker run --name [nome-amigavel] [imagem]
```

Atribui um nome fácil de lembrar, simplificando comandos futuros (`stop`, `logs`, etc.).

Listar contêineres ativos (`ps`)

```
docker ps
```

Mostra apenas os contêineres que estão em execução no momento, com seus IDs, nomes e status.

Inspeccionando e Depurando Contêineres

Listar Todos

```
docker ps -a
```

O flag `-a` (all) mostra todos os contêineres, incluindo os que foram parados. Crucial para investigar por que um contêiner falhou ao iniciar.

Acessar o Terminal

```
docker exec -it [contêiner] /bin/bash
```

Abre uma sessão de shell interativa (`-it`) dentro de um contêiner em execução para explorar seu sistema de arquivos e processos.

Ver Logs

```
docker logs -f [contêiner]
```

Exibe a saída de um contêiner. O flag `-f` (follow) transmite os logs em tempo real.

Inspeccionar Metadados

```
docker inspect [contêiner]
```

Retorna um JSON detalhado com toda a configuração do contêiner, incluindo endereços IP, volumes montados e variáveis de ambiente.

Boas Práticas: Limpeza e Manutenção do Ambiente

Contêineres e imagens parados consomem espaço em disco. Manter seu ambiente limpo é uma prática essencial de 'higiene digital'.

Parar um Contêiner

```
docker stop [ID ou nome]
```

Envia um sinal de parada para o processo, permitindo um desligamento gracioso.

Remover um Contêiner

```
docker rm [ID ou nome]
```

Exclui permanentemente um contêiner parado.

Limpeza Automática

```
docker container prune
```

Um comando poderoso que remove **todos** os contêineres parados de uma só vez. Use com cuidado.



Workflow Completo: Do Pull à Limpeza com NGINX

Vamos baixar uma imagem do NGINX, executá-la em background, inspecioná-la e, finalmente, limpar todos os recursos.

1. Baixar a imagem específica e leve do NGINX

```
docker pull nginx:alpine
```

2. Executar o contêiner em modo detached, nomeá-lo e mapear a porta

```
docker run -d --name meu-nginx -p 8080:80 nginx:alpine
```

3. Verificar se o contêiner está em execução

```
docker ps
```

4. Acompanhar os logs de acesso (pressione Ctrl+C para sair)

```
docker logs -f meu-nginx
```

5. Acessar o terminal dentro do contêiner para explorar

```
docker exec -it meu-nginx /bin/sh
```

6. Parar e remover o contêiner

```
docker stop meu-nginx
```

```
docker rm meu-nginx
```

7. Remover a imagem que não está mais em uso

```
docker rmi nginx:alpine
```

Próximos Passos: Construindo Suas Próprias Imagens

Você agora tem a base sólida para trabalhar com Docker no dia a dia. O próximo passo é passar de consumidor de imagens para criador.



Dominando o `Dockerfile`: Aprenda a escrever suas próprias "receitas" para criar imagens customizadas, definindo cada camada e configuração da sua aplicação.



Otimização e Camadas: Entenda como o Docker utiliza o cache de camadas para construir imagens de forma eficiente e as técnicas para criar imagens menores e mais seguras.

Pratique os comandos aprendidos e prepare-se para o próximo nível: automatizar a criação dos seus ambientes de desenvolvimento.

